

JavaScript

DOM (Document Object Model)

Laboratorio de Programación

UNPA – UACO

Ing. Jose Rasjido

AGENDA

- ✓ **Introducción al DOM**
- ✓ Manipulación del DOM
- ✓ API classList
- ✓ API dataset
- ✓ Bibliografía

Introducción al DOM

¿Qué es DOM?

Es una API para la representación y manipulación del contenido de los documentos HTML y XML.

Cada uno de los elementos de un documento, son representados en el DOM mediante un árbol de objetos.

Introducción al DOM

Considere el siguiente documento HTML:

```
<html>

  <head>

    <title>Sample Document</title>

  </head>

  <body>

    <h1>An HTML Document</h1>

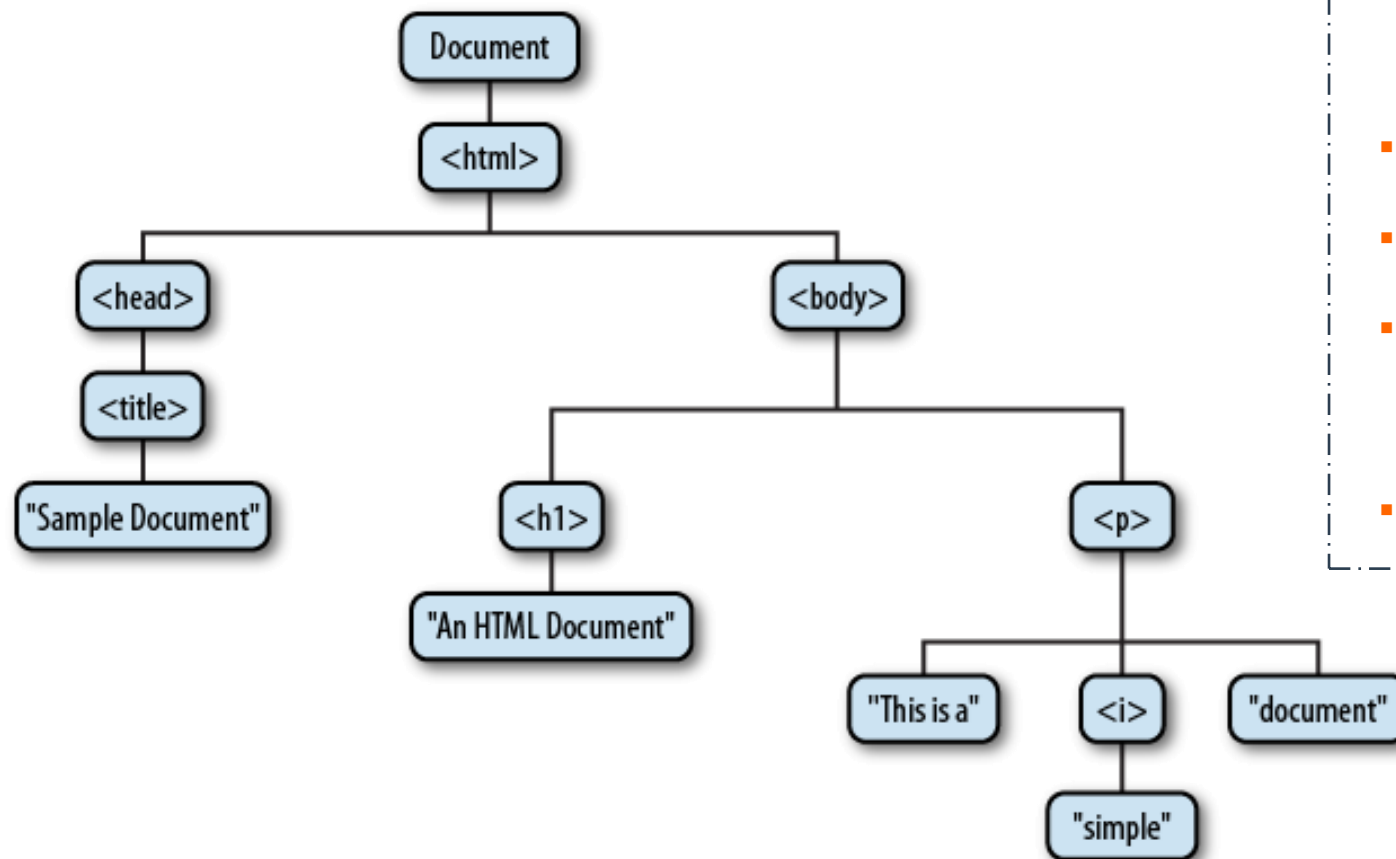
    <p>This is a <i>simple</i> document.</p>

  </body>

</html>
```

Introducción al DOM

Representación DOM del documento anterior:

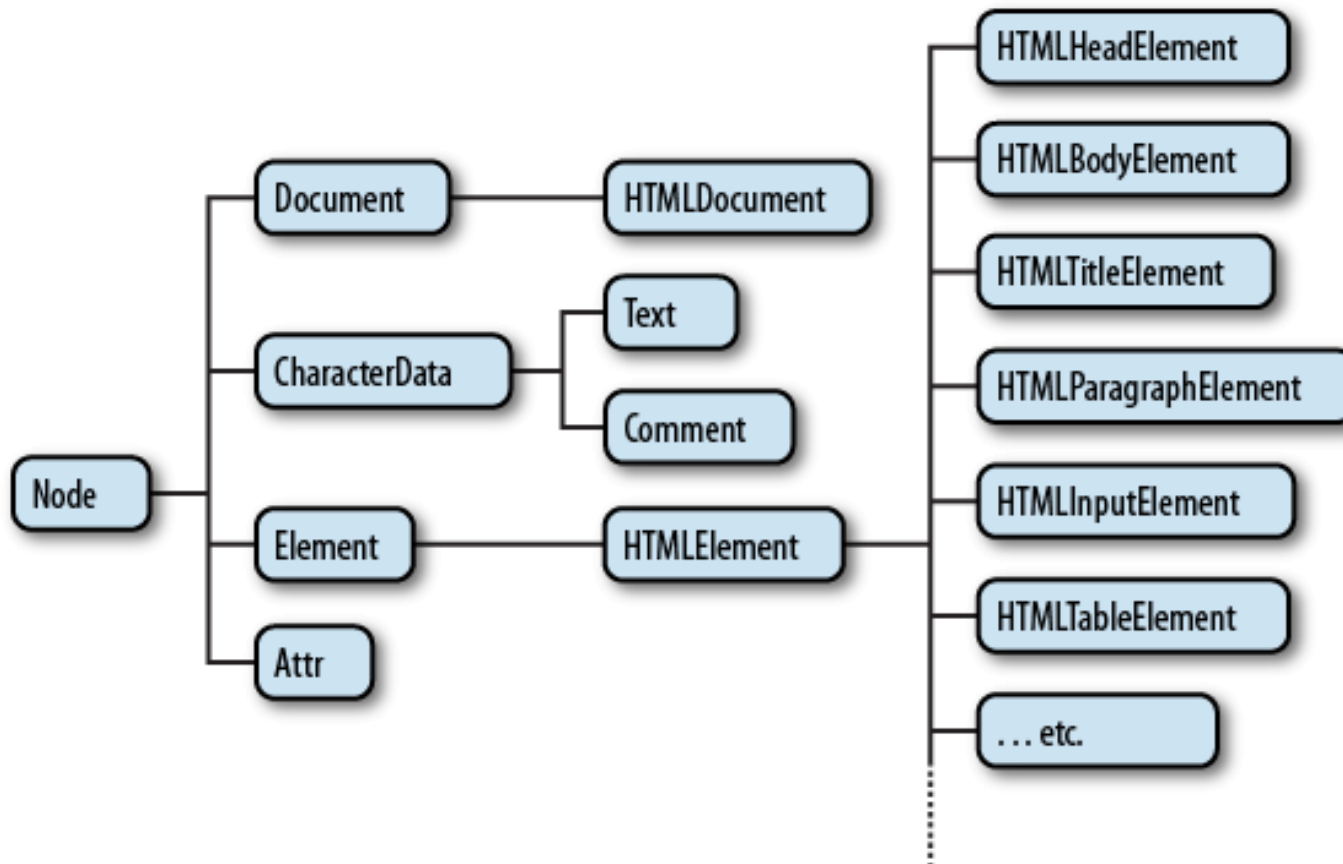


Terminología

- Nodos padres
- Nodos hermanos
- Nodos descendientes
- Nodos ancestros

Introducción al DOM

Cada nodo del árbol es representado por un objeto del tipo *Node*.



La figura muestra una jerarquía de clase parcial de los nodos.

Introducción al DOM

➤ Seleccionando los elementos del documento

Para acceder al árbol del documento, se hace mediante la variable global **document**.

El DOM define varias formas de acceder a los elementos:

- Mediante el atributo **id**
- Mediante el atributo **name**
- Mediante el nombre de la etiqueta
- Mediante la clase CSS o un selector CSS

Introducción al DOM

Seleccionar mediante id - `getElementById()`

El elemento debe tener un identificador único asignado mediante el atributo `id`.

```
let menu = document.getElementById("ul-menu");
```

Devuelve un objeto del tipo *Node*.

Devuelve `null`, si no se encontró el elemento.

Introducción al DOM

Seleccionar mediante **name** - **getElementByName()**

El elemento debe tener un nombre asignado mediante el atributo **name**.

Este identificador puede repetirse en el documento, por lo que la consulta puede devolver mas de un elemento.

```
let radiobuttons = document.getElementsByName("colores-favoritos");
```

Devuelve un objeto del tipo **NodeList**.

Devuelve **null** si no hubo coincidencias.

Introducción al DOM

Seleccionar a través de selectores `querySelector()`

Devuelve el primer elemento del documento, que coincida con el grupo especificado de selectores.

```
let element = document.querySelector(selectores);
```

Devuelve el primer elemento encontrado.

Devuelve **null** si no se encuentran elementos.

Introducción al DOM

Seleccionar a través de selectores `querySelectorAll()`

Devuelve una lista de elementos, que coinciden con el grupo especificado de selectores.

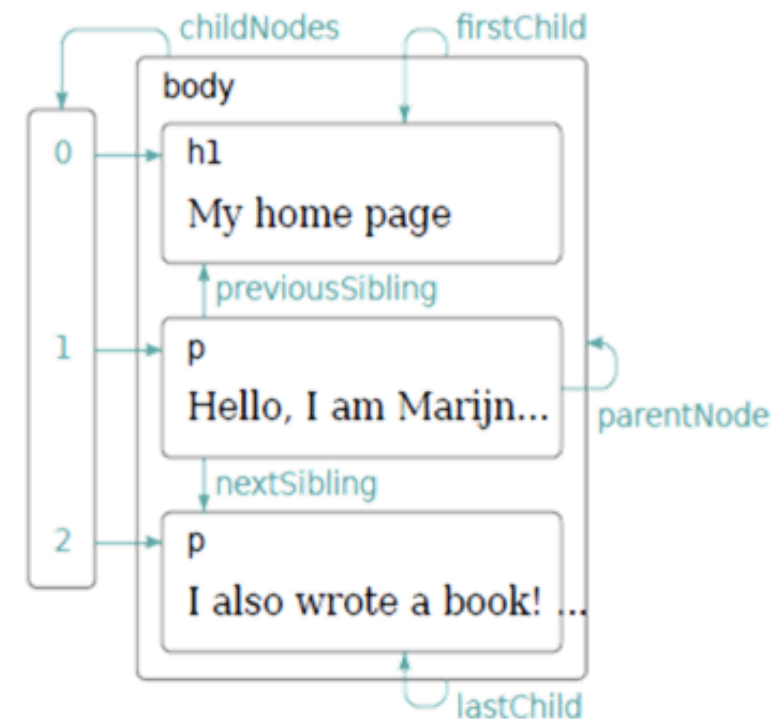
```
let element = document.querySelectorAll(selectores);
```

Devuelve un `NodeList`.

Introducción al DOM

➤ Propiedades para recorrer los nodos cercanos

- **parentNode**. Retorna el nodo padre del actual, o **null** si no lo tiene.
- **childNodes**. Devuelve un array de solo lectura (NodeList) con los hijos directos.
- **firstChild** - **lastChild**. El primer y último hijo de un nodo, o null si no tiene hijos.
- **nextSibling** - **previousSibling**. El siguiente y anterior nodo hermano del actual.



AGENDA

- ✓ Introducción al DOM
- ✓ **Manipulación del DOM**
- ✓ API classList
- ✓ API dataset
- ✓ Bibliografía

Manipulación del DOM

➤ Insertar nodos o elementos al DOM

Existen diferentes formas, algunas de ellas son:

//Inserta el código HTML en la posición “pos”

```
nodo.insertAdjacentElement(pos, elementNode);
```

```
nodo.insertAdjacentHTML(pos, str);
```

```
nodo.insertAdjacentText(pos, textNode);
```

Manipulación del DOM

Para las tres versiones *adjacentHTML*, hay 4 posiciones posibles para insertar:

- **beforebegin**: El elemento se inserta **antes** de la etiqueta HTML de apertura.
- **afterbegin**: El elemento se inserta **dentro** de la etiqueta HTML, antes de su primer hijo.
- **beforeend**: El elemento se inserta **dentro** de la etiqueta HTML, después de su último hijo. = **appendChild()**
- **afterend**: el elemento se inserta **después** de la etiqueta HTML de cierre.

Manipulación del DOM

➤ Eliminar y reemplazar nodos

- Eliminar un nodo:

```
nodo.remove()
```

- Eliminar un nodo hijo:

```
nodo.removeChild(target);
```

```
nodo.parentNode.removeChild(nodo); ¿?
```

- Reemplazar un nodo por otro:

```
nodo.replaceChild(nodoNuevo, nodoViejo);
```


AGENDA

- ✓ Introducción al DOM
- ✓ Manipulación del DOM
- ✓ **API classList**
- ✓ API dataset
- ✓ Bibliografía

API classList

Element.classList

Es una propiedad de los elementos **HTML**, y devuelve una colección activa de los atributos de clase del elemento.

Es una forma práctica de acceder a la lista de clases de un elemento, como una cadena de texto delimitada por espacios a través de **element.className**.

API classList

➤ Métodos

- `add(String [, String])`. Añade las clases indicadas. Se ignoran repetidas.
- `remove(String [, String])`. Elimina las clases indicadas.
- `item(Number)`. Devuelve el valor de la clase, según el índice indicado.

API classList

- `toggle(String [, force])`. Alterna el valor de la clase. Si la clase existe, la elimina. Sino la agrega. Si *force* esta presente y es true, se agrega. Sino, si es false se elimina.
- `contains(String)`. Comprueba si la clase existe en el elemento.
- `replace(oldClass, newClass)`. Reemplaza una clase existente por una nueva.

AGENDA

- ✓ Introducción al DOM
- ✓ Manipulación del DOM
- ✓ API classList
- ✓ **API dataset**
- ✓ Bibliografía

API dataset

➤ `HTMLElement.dataset`

Es una propiedad de los elementos **HTML**, y provee acceso de lectura y escritura para los atributos personalizados del tipo **data-***.

En **HTML**, los nombres de los atributos comienzan con **data-**

- Pueden contener letras, números, guiones medios, punto, dos puntos y guiones bajos.
- Cualquier letra capital (A..Z) es convertida a minúsculas.

API dataset

➤ HTMLElement.dataset

En **JAVASCRIPT**, los atributos personalizados se acceden mediante la propiedad **dataset**. Además, para los atributos personalizados:

- Se omite **data-**
- Se omiten los guiones
- Se convierte el texto a la notación *camelCased*

AGENDA

- ✓ Introducción al DOM
- ✓ Manipulación del DOM
- ✓ API classList
- ✓ API dataset
- ✓ **Bibliografía**

Bibliografía

- **Estándar ECMAScript, Ecma-262**
 - <https://www.ecma-international.org/publications/standards/Ecma-262.htm>
- **Documentación de Mozilla**
 - <https://developer.mozilla.org/es/docs/Web/JavaScript/Guide>
- **JavaScript – The Definitive Guide**
 - David Flanagan - O'REILLY
 - ISBN: 978-0-596-80552-4